

Global Temperature Anomalies in Practice

An Open, Reproducible Framework for Baseline Harmonization and ENSO-Aware Visualization

J. Eduardo Vera-Valdés
Aalborg University, CoRE
eduardo@math.aau.dk

2026-04-27

Abstract This paper develops a processing pipeline to download and process global temperature anomaly data from multiple datasets. The workflow downloads and compiles five of the most widely used global temperature anomaly datasets: HadCRUT5, GISTEMP, NOAAGlobalTemp, Berkeley Earth, and ERA5. The data is processed to calculate the temperature anomalies relative to the 1850-1900 baseline, the preindustrial level, making it easy to compare the datasets. The paper also plots the data and highlight El Niño and La Niña events using the ONI dataset. This makes it possible to visually inspect the relationship between global temperature anomalies and El Niño and La Niña events. The compiled dataset is saved in a CSV file and is directly accessible via the paper’s repository.

1 Introduction

Global temperature anomalies are central to climate variability and change analysis, but practical use across datasets is often limited by inconsistent baseline periods and heterogeneous data formats. These differences reduce comparability, increase processing effort, and weaken reproducibility in applied climate research and services.

This paper addresses that operational gap by providing a concise, reproducible setup that harmonizes major global temperature anomaly datasets to a common preindustrial baseline (1850-1900) and a common tabular structure. The workflow is designed for direct reuse, supporting Open Data and Open Science while improving usability for climate services and practitioner workflows. The codebase is implemented in Julia [1] to combine high readability with fast execution.

We compile and standardize five widely used datasets: HadCRUT5 [2], GISTEMP [3], NOAAGlobalTemp [4], Berkeley Earth [5], and ERA5 [6]. To strengthen interpretation, we integrate the Oceanic Nino Index [7], [8] and annotate El Nino and La Nina phases in the visualizations, making short-term ENSO variability easier to interpret alongside long-term warming.

The remainder of the notebook is organized as follows: we load the required packages and helper functions, process each dataset, construct a unified dataset, and present comparative plots with ENSO annotations. The compiled dataset is saved as a CSV file in the repository. The codebase is structured for reproducibility and reuse through clear documentation and modular functions that can be adapted to other datasets or baseline definitions.

2 Necessary Packages and Functions

To ensure reproducibility and ease of use, this notebook uses a Julia project environment with pinned dependencies. The environment is defined by `Project.toml` (and `Manifest.toml` when present), allowing others to reproduce the same setup by activation. In practice, Julia's environment system makes reproducibility straightforward, while Python workflows can be less consistent when dependencies are not pinned.

```
# Initialize the project environment
```

```
using Pkg
Pkg.activate(pwd())
```

```
Activating project at `~/Library/CloudStorage/OneDrive-AalborgUniversitet/Research/CLIMATE/Global Temperature Anomalies`
```

The code above activates the project environment in the current directory. We then load the packages used for data download, processing, and visualization: `Dates`, `CSV`, `DataFrames`, `HTTP`, `CDSAPI`, `Statistics`, `NCDatasets`, and `Plots`. `Dates` and `Statistics` are included in the Julia standard library; the remaining packages can be installed with `Pkg.add`:

```
# Install necessary packages
```

```
Pkg.add(["Plots", "CSV", "DataFrames", "HTTP", "CDSAPI", "NCDatasets"])
```

```
Resolving package versions...
```

```
Project No packages added to or removed from `~/Library/CloudStorage/OneDrive-AalborgUniversitet/Research/CLIMATE/Global Temperature Anomalies/Project.toml`
```

```
Manifest No packages added to or removed from `~/Library/CloudStorage/OneDrive-AalborgUniversitet/Research/CLIMATE/Global Temperature Anomalies/Manifest.toml`
```

After installation, packages are loaded with `using`.

```
# Load the necessary packages
```

```
using Statistics, Dates, CSV, DataFrames, HTTP, CDSAPI, NCDatasets, Plots
```

The next utility converts wide-format data to long format. This is required for `GISTEMP`, where each row corresponds to one year and monthly values are stored in separate columns. The `longseries` function reshapes this structure into one row per month.

```
# Function to convert wide format data to long format

function longseries(data)
    height = size(data, 1) # Number of rows, equivalent to the number of years
    last_row = 12 - count(ismissing, data[end, 2:13]) # Number of non-missing months
    in the last year

    many = (height - 1) * 12 + last_row # Total number of months in the long format
    long = zeros(many, 1) # Long format array

    for ii = 1:(height-1) # Loop through all years except the last one
        for jj = 1:12 # Loop through all months
            long[(ii-1)*12+jj] = data[ii, jj+1]
        end
    end

    for jj = 1:last_row # Loop through the last year
        long[(height-1)*12+jj] = data[height, jj+1]
    end

    return long
end
```

```
longseries (generic function with 1 method)
```

The `longseries` function takes a `DataFrame` in wide format and returns a long-format array. It calculates the total number of months based on the number of years, same as the number of rows, and the number of non-missing months in the last year. It then fills the long-format array by looping through the years and months.

3 HadCRUT5

The HadCRUT5 dataset is a global monthly average temperature dataset compiled by the Met Office Hadley Centre and the Climatic Research Unit at the University of East Anglia [2]. It is one of the most widely used datasets for global temperature anomalies. The HadCRUT5 dataset is available in CSV format from the Met Office website. The code below downloads the data, processes it, and saves it in a CSV file. The data is then used to calculate the temperature anomalies relative to the 1850-1900 baseline.

```
# Download the HadCRUT temperature data
# URL of the HadCRUT5 global monthly average CSV
```

```

hurl = "https://www.metoffice.gov.uk/hadobs/hadcrut5/data/HadCRUT.5.1.0.0/analysis/
diagnostics/HadCRUT.5.1.0.0.analysis.summary_series.global.monthly.csv"
# Local filename to save
hfilename = "data/HadCRUT5_global_monthly_average.csv"
open(hfilename, "w") do io
    write(io, HTTP.get(hurl).body)
end

rawhadcrut = CSV.read(hfilename, DataFrame)
rename!(rawhadcrut, :Time => :Date)
rename!(rawhadcrut, "Anomaly (deg C)" => :RawTemperature)

hadcrut = rawhadcrut[:, [:Date, :RawTemperature]]

oldbase = mean(hadcrut[(hadcrut.Date.>=Date(1850, 1, 1)).&(hadcrut.Date.<Date(1900, 1,
1)), :RawTemperature])
hadcrut[:, :Temp] = hadcrut[:, :RawTemperature] .- oldbase;

CSV.write(hfilename, hadcrut)

last(hadcrut, 5) # Show the first 5 rows of the HadCRUT5 data

```

5×3 DataFrame

Row	Date	RawTemperature	Temp
	Date	Float64	Float64
1	2025-10-01	1.09011	1.45001
2	2025-11-01	1.0748	1.43471
3	2025-12-01	0.924534	1.28444
4	2026-01-01	0.997841	1.35775
5	2026-02-01	1.04689	1.4068

The HadCRUT5 dataset is now saved here `HadCRUT5_global_monthly_average.csv`. The data contains the date, raw temperature anomalies, and the temperature anomalies relative to the 1850-1900 baseline.

4 GISTEMP

The GISTEMP dataset is a global monthly average temperature dataset [3]. It is available in CSV format from the NASA GISS website. The code below downloads the data, processes it, and saves it in a CSV file. The data is then used to calculate the temperature anomalies relative to the 1850-1900 baseline.

Note that the GISTEMP data is in a wide format, with a column for each month. We will convert it to a long format, where each row corresponds to a single month using the `longseries` function defined above.

Moreover, the GISTEMP data starts in 1880, see below. For consistency, the temperature anomalies are calculated relative to the 1850-1900 baseline. Following the data source recommendation, to calculate the GISTEMP anomaly with respect to 1850-1900, we can adjust to 1880-1899 from 1951-1980, and then make a small adjustment of 0.038°C to account for the pre-1880 data.

```
# Download the GISTEMP temperature data
# URL of the GISTEMP global monthly average CSV
gurl = "https://data.giss.nasa.gov/gistemp/tabledata_v4/GLB.Ts%2BdSST.csv"
# Local filename to save
gfilename = "data/GISTEMP_global_monthly_average.csv"
# Download the file

open(gfilename, "w") do io
    write(io, HTTP.get(gurl).body)
end

longgistemp = CSV.read(gfilename, DataFrame, header=2, missingstring=["***"])
gistemp = longseries(longgistemp)[: ]
Tt = length(gistemp) - 1

start = Date(1880, 1, 1) # Start date of the dataset
fin = start + Month(Tt) # End date of the dataset
fechas = collect(start:Month(1):fin) # Create a Date array

gistemp = DataFrame(:Date=>fechas, :RawTemp=>gistemp)

oldbase = mean(gistemp[(gistemp.Date.>=Date(1880, 1, 1)).&(gistemp.Date.<Date(1900, 1, 1)), :RawTemp])
gistemp[!, :Temp] = gistemp[!, :RawTemp] .- oldbase .+ 0.038 # Adjust for pre-1880 data

CSV.write(gfilename, gistemp)

last(gistemp, 5) # Show the first 5 rows of the GISTEMP data
```

5×3 DataFrame

Row	Date	RawTemp	Temp
	Date	Float64	Float64
1	2025-11-01	1.2	1.46287
2	2025-12-01	1.07	1.33288
3	2026-01-01	1.08	1.34288
4	2026-02-01	1.24	1.50287
5	2026-03-01	1.28	1.54287

The GISTEMP dataset is now saved here [GISTEMP_global_monthly_average.csv](#). The data contains the date, raw temperature anomalies, and the temperature anomalies relative to the 1850-1900 baseline.

5 NOAAGlobalTemp

The NOAAGlobalTemp dataset is a global monthly average temperature dataset compiled by the National Oceanic and Atmospheric Administration (NOAA) [4]. It is available in CSV format from the NOAA NCEI website. The code below downloads the data, processes it, and saves it in a CSV file. The data is then used to calculate the temperature anomalies relative to the 1850-1900 baseline.

Note that the url changes every month as new data is added: note the `...YYYYMM.asc` name. If the code does not work in the future, try updating the url using the latest year and month.

```
# Download the NOAAGlobalTemp temperature data
# URL of the NOAAGlobalTemp global monthly average CSV ---yearmonth.asc
nurl = "https://www.ncei.noaa.gov/data/noaa-global-surface-temperature/v6.1/access/
timeseries/aravg.mon.land_ocean.90S.90N.v6.1.0.202603.asc"
# Local filename to save
nfilename = "data/NOAA_global_monthly_average.csv"

# Download the file
open(nfilename, "w") do io
    write(io, HTTP.get(nurl).body)
end

lines = readlines(nfilename)
cleaned_lines = [join(split(strip(line)), ",") for line in lines]

# Write to file
write(nfilename, join(cleaned_lines, "\n"))

rawnoaa = CSV.read(nfilename, DataFrame; delim=',', header=0)

fechas = Date.(rawnoaa.Column1, rawnoaa.Column2, 1) # Create Date column from Column1
and Column2

noaa = DataFrame(:Date=>fechas, :RawTemp=>rawnoaa.Column3)

oldbase = mean(noaa[(noaa.Date.>=Date(1850, 1, 1)).&(noaa.Date.<Date(1900, 1,
1)), :RawTemp])
noaa[:, :Temp] = noaa[:, :RawTemp] .- oldbase

CSV.write(nfilename, noaa)

last(noaa, 5) # Show the first 5 rows of the NOAAGlobalTemp data
```

5x3 DataFrame

Row	Date	RawTemp	Temp
	Date	Float64	Float64
1	2025-11-01	0.518696	1.29869
2	2025-12-01	0.44877	1.22877
3	2026-01-01	0.481615	1.26161
4	2026-02-01	0.516442	1.29644
5	2026-03-01	0.636699	1.4167

The NOAAGlobalTemp dataset is now saved here `NOAA_global_monthly_average.csv`. The data contains the date, raw temperature anomalies, and the temperature anomalies relative to the 1850-1900 baseline.

6 Berkeley Earth

The Berkeley Earth dataset is a global monthly average temperature dataset [5]. It is available in CSV format from the Berkeley Earth website. The code below downloads the data, processes it, and saves it in a CSV file. The data is then used to calculate the temperature anomalies relative to the 1850-1900 baseline.

```
# Download the Berkeley Earth temperature data
# URL of the Berkeley Earth global monthly average CSV
burl = "https://storage.googleapis.com/berkeley-earth-temperature-hr/global/Global_
TAVG_monthly.txt"
# Local filename to save
bfilename = "data/BerkeleyEarth_global_monthly_average.csv"
# Download the file
open(bfilename, "w") do io
    write(io, HTTP.get(burl).body)
end

rawtemp = CSV.read(bfilename, DataFrame, comment="%", delim=" ", ignorerepeated=true)

colnames = [:Year, :Month, :Anomaly_Monthly, :Unc_Monthly,
            :Anomaly_Annual, :Unc_Annual, :Anomaly_5yr, :Unc_5yr,
            :Anomaly_10yr, :Unc_10yr, :Anomaly_20yr, :Unc_20yr]
rename!(rawtemp, colnames)

rawtemp.Date = Date.(rawtemp.Year, rawtemp.Month, 1) # Create Date column from Year
and Month
rename!(rawtemp, :Anomaly_Monthly => :RawTemperature)

berkeley = rawtemp[!, [:Date, :RawTemperature]]
```

```

oldbase = mean(rawtemp[(rawtemp.Date.>=Date(1850, 1, 1)).&(rawtemp.Date.<Date(1900, 1,
1)), :RawTemperature])
rawtemp[!, :Temp] = rawtemp[!, :RawTemperature] .- oldbase

berkeley.Temp = rawtemp.Temp

CSV.write(bfilename, berkeley)

last(berkeley, 5) # Show the first 5 rows of the Berkeley Earth data

```

5x3 DataFrame

Row	Date	RawTemperature	Temp
	Date	Float64	Float64
1	2025-11-01	1.21	1.45052
2	2025-12-01	1.089	1.32952
3	2026-01-01	1.142	1.38252
4	2026-02-01	1.234	1.47452
5	2026-03-01	1.215	1.45552

The Berkeley Earth dataset is now saved here `BerkeleyEarth_global_monthly_average.csv`. The data contains the date, raw temperature anomalies, and the temperature anomalies relative to the 1850-1900 baseline.

7 ERA5

ERA5 is the fifth generation ECMWF reanalysis for the global climate and weather from 1940 onwards [6]. Data is available from 1940 onwards. ERA5 replaces the ERA-Interim reanalysis. The ERA5 dataset is available in NetCDF format from the Copernicus Climate Data Store. The code to download and process the ERA5 data is shown below.

Note that the API requires setting up a personal key in a `.cdsapirc` file, which should be placed in the root directory of the project. You can get a key by requesting it at the Copernicus Climate Data Store website. The code below assumes that you have already set up the `.cdsapirc` file with your key.

The code below uses the `CDSAPI` package to download the data in a NetCDF file. The data is large, close to 1.5 GB, so it may take some time to download.

```

# Download the ERA5 temperature data

dataset = "reanalysis-era5-single-levels-monthly-means"

request = "{
  \"product_type\": [\"monthly_averaged_reanalysis\"],
  \"variable\": [\"2m_temperature\"],

```



```

    "year": [
        "1940", "1941", "1942", "1943", "1944", "1945",
        "1946", "1947", "1948", "1949", "1950", "1951",
        "1952", "1953", "1954", "1955", "1956", "1957",
        "1958", "1959", "1960", "1961", "1962", "1963",
        "1964", "1965", "1966", "1967", "1968", "1969",
        "1970", "1971", "1972", "1973", "1974", "1975",
        "1976", "1977", "1978", "1979", "1980", "1981",
        "1982", "1983", "1984", "1985", "1986", "1987",
        "1988", "1989", "1990", "1991", "1992", "1993",
        "1994", "1995", "1996", "1997", "1998", "1999",
        "2000", "2001", "2002", "2003", "2004", "2005",
        "2006", "2007", "2008", "2009", "2010", "2011",
        "2012", "2013", "2014", "2015", "2016", "2017",
        "2018", "2019", "2020", "2021", "2022", "2023",
        "2024", "2025", "2026"
    ],
    "month": [
        "01", "02", "03",
        "04", "05", "06",
        "07", "08", "09",
        "10", "11", "12"
    ],
    "time": ["00:00"],
    "data_format": "netcdf",
    "download_format": "unarchived"
}"""

CDSAPI.retrieve(dataset, request, "data/era5_2m_temperature_2020.nc")

```

Once the data is downloaded, we can use the `NCdatasets` package to read the NetCDF file and process the data. The code below reads the data, extracts the latitude, longitude, time, and temperature variables, and calculates the global average temperature anomaly for each month.

```

filename = "data/era5_2m_temperature.nc";
ds = NCdataset(filename, "r");

lat_era = ds["latitude"][:];
lon_era = ds["longitude"][:];
time_era = ds["valid_time"][:];
date_era = Date.(time_era)
lat = lat_era *  $\pi/180$ 

```

Processing the data is computationally intensive, so we will process the data in chunks of 120 months (10 years) to avoid memory issues. The code below calculates the global average temperature anomaly for each month and saves it in a CSV file. The temperature anomalies are calculated relative to the 1850-1900 baseline, which is estimated to be 0.88°C above the preindustrial level according to the latest ERA5 reanalysis [6].

```
ini = 1
chunk_size = 120 # Number of months to skip for plotting

era5_chunk = DataFrame()
global_mean_C = zeros(length(date_era))

while ini < length(date_era)

  for i in ini:(ini+chunk_size-1)
    t2m = ds["t2m"][ :, :, i] # 2D slice [lon, lat] for first time; in K
    t_lon_mean = mean(t2m, dims=1) # [lat]; drops new dim=2
    weights = cos.(lat)
    global_mean_K = sum(t_lon_mean' .* weights) / sum(weights)
    global_mean_C[i] = global_mean_K - 273.15
  end

  display(global_mean_C[ini:(ini+chunk_size-1)])

  ini = ini + chunk_size

end
```

```
display(global_mean_C)

era5_raw = DataFrame(Date=date_era, RawTemperature=global_mean_C)

CSV.write("data/ERA5_global_monthly_average_raw.csv", era5_raw)
```

Once the monthly global mean temperature anomalies are calculated, we can calculate the temperature anomalies relative to the 1850-1900 baseline. The code below calculates the monthly climatology for the period 1991-2020 and then calculates the anomalies relative to this climatology. Finally, we add the offset to convert to preindustrial anomalies and save the data in a CSV file.

```
time_dates = Date.(ds["valid_time"][ :]) # If not already
start_1991 = findfirst(≥(Date(1991,1,1)), time_dates)
end_2020 = findfirst(≥(Date(2021,1,1)), time_dates) - 1
n_clim = end_2020 - start_1991 + 1 # ~360
```

```

month_clim = zeros(12)
for m in 1:12
    idx = (month.(time_dates[start_1991:end_2020]) .== m)
    month_clim[m] = mean(global_mean_C[start_1991:end_2020][idx])
end

anom_1991_2020 = global_mean_C .- month_clim[month.(time_dates)]
offset_preind = 0.88 # Latest ERA5-to-1850-1900; check HadCRUT5 [web:73]
pa_anom = anom_1991_2020 .+ offset_preind

println("2025 annual PA anomaly: ", round(mean(pa_anom[end-11:end]), digits=2), "°C")

era5 = DataFrame(Date=time_dates, RawTemperature=anom_1991_2020, Temp=pa_anom)
CSV.write("data/ERA5_global_monthly_average.csv", era5)

```

The snippet below reads the processed ERA5 data from the CSV file and shows the last 5 rows of the data.

```

erafilename = "data/ERA5_global_monthly_average.csv"
era5 = CSV.read(erafilename, DataFrame)

```

```

1034×3 DataFrame
  Row | Date          RawTemperature  Temp
      | Date          Float64         Float64
-----|-----
  1 | 1940-01-01    -0.944512  -0.064512
  2 | 1940-02-01    -0.856925   0.023075
  3 | 1940-03-01    -0.885271  -0.00527081
  4 | 1940-04-01    -0.605069   0.274931
  5 | 1940-05-01    -0.515772   0.364228
  6 | 1940-06-01    -0.667846   0.212154
  7 | 1940-07-01    -0.450324   0.429676
  8 | 1940-08-01    -0.498395   0.381605
  ⋮  | ⋮             ⋮             ⋮
1028 | 2025-08-01     0.488983   1.36898
1029 | 2025-09-01     0.661873   1.54187
1030 | 2025-10-01     0.695328   1.57533
1031 | 2025-11-01     0.648296   1.5283
1032 | 2025-12-01     0.486227   1.36623
1033 | 2026-01-01     0.511353   1.39135
1034 | 2026-02-01     0.531408   1.41141
      1019 rows omitted

```

```
last(era5, 5) # Show the first 5 rows of the ERA5 data
```

5x3 DataFrame

Row	Date	RawTemperature	Temp
	Date	Float64	Float64
1	2025-10-01	0.695328	1.57533
2	2025-11-01	0.648296	1.5283
3	2025-12-01	0.486227	1.36623
4	2026-01-01	0.511353	1.39135
5	2026-02-01	0.531408	1.41141

8 Oceanic Niño Index (ONI)

El Niño (La Niña) is a phenomenon in the equatorial Pacific Ocean characterized by a five consecutive 3-month running mean of sea surface temperature anomalies in the Niño 3.4 region that is above (below) the threshold of $+0.5^{\circ}\text{C}$ (-0.5°C).

The data is obtained from the Extended Reconstructed Sea Surface Temperature (ERSST) dataset, which is a global monthly analysis of sea surface temperature data derived from the International Comprehensive Ocean–Atmosphere Dataset (ICOADS) [7], [8]. The ONI data is available in CSV format from the NOAA Climate Monitoring website. The code below downloads the data, processes it, and saves it in a CSV file.

```
# Download the ONI data
ourl = "https://www.cpc.ncep.noaa.gov/data/indices/sstoi.indices"
ofilename = "data/Nino_data.csv"

open(ofilename, "w") do io
    write(io, HTTP.get(ourl).body)
end

lines = readlines(ofilename)
cleaned_lines = [join(split(strip(line)), ",") for line in lines]

# Write to file
write(ofilename, join(cleaned_lines, "\n"))

rawoni = CSV.read(ofilename, DataFrame; delim=',', header=1)
fechas = Date.(rawoni.YR, rawoni.MON, 1) # Create Date column from YR and MON

oni = DataFrame(Date=fechas, Anom=rawoni[:, :ANOM_3])

CSV.write(ofilename, oni)
```

```
last(oni, 5) # Show the first 5 rows of the ONI data
```

5x2 DataFrame

Row	Date	Anom
	Date	Float64
1	2025-11-01	-0.68
2	2025-12-01	-0.61
3	2026-01-01	-0.54
4	2026-02-01	-0.2
5	2026-03-01	0.03

The ONI dataset is now saved here `Nino_data.csv`. The data contains the date and the ONI anomalies.

9 Merge all datasets

The code below merges all the datasets into a single dataset. It uses the `leftjoin` function to merge the datasets on the `Date` column. The resulting dataset contains the temperature anomalies for each dataset, as well as the ONI data. The merged dataset is saved in a CSV file.

```
# Load the datasets
hfilename = "data/HadCRUT5_global_monthly_average.csv"
gfilename = "data/GISTEMP_global_monthly_average.csv"
nfilename = "data/NOAA_global_monthly_average.csv"
bfilename = "data/BerkeleyEarth_global_monthly_average.csv"
erafilename = "data/ERA5_global_monthly_average.csv"
ofilename = "data/Nino_data.csv"

hadcrut = CSV.read(hfilename, DataFrame)
gistemp = CSV.read(gfilename, DataFrame)
noaa = CSV.read(nfilename, DataFrame)
berkeley = CSV.read(bfilename, DataFrame)
era5 = CSV.read(erafilename, DataFrame)
oni = CSV.read(ofilename, DataFrame)

# Dates
min_date = minimum([minimum(hadcrut.Date), minimum(gistemp.Date), minimum(noaa.Date),
minimum(berkeley.Date), minimum(oni.Date)])
max_date = maximum([maximum(hadcrut.Date), maximum(gistemp.Date), maximum(noaa.Date),
maximum(berkeley.Date), maximum(oni.Date)])
complete_dates = collect(min_date:Month(1):max_date)
compiled_data = DataFrame(Date=complete_dates)
```

```

# HadCRUT5
compiled_data = leftjoin(compiled_data, hadcrut, on = :Date)
rename!(compiled_data, :RawTemperature => :HadCRUT_RawTemperature)
rename!(compiled_data, :Temp => :HadCRUT_Temp)
sort!(compiled_data, :Date)

# GISTEMP
compiled_data = leftjoin(compiled_data, gistemp, on = :Date)
rename!(compiled_data, :RawTemp => :GISTEMP_RawTemperature)
rename!(compiled_data, :Temp => :GISTEMP_Temp)
sort!(compiled_data, :Date)

# NOAA
compiled_data = leftjoin(compiled_data, noaa, on = :Date)
rename!(compiled_data, :RawTemp => :NOAA_RawTemperature)
rename!(compiled_data, :Temp => :NOAA_Temp)
sort!(compiled_data, :Date)

# Berkeley Earth
compiled_data = leftjoin(compiled_data, berkeley, on = :Date)
rename!(compiled_data, :RawTemperature => :Berkeley_RawTemperature)
rename!(compiled_data, :Temp => :Berkeley_Temp)
sort!(compiled_data, :Date)

# ERA5
compiled_data = leftjoin(compiled_data, era5, on = :Date)
rename!(compiled_data, :RawTemperature => :ERA5_RawTemperature)
rename!(compiled_data, :Temp => :ERA5_Temp)
sort!(compiled_data, :Date)

# ONI
compiled_data = leftjoin(compiled_data, oni, on = :Date)
rename!(compiled_data, :Anom => :ONI_Anomaly)
sort!(compiled_data, :Date)

# Save the compiled data
compiled_filename = "data/Compiled_Global_Temperature_Data.csv"
CSV.write(compiled_filename, compiled_data)

last(compiled_data, 5) # Show the last 5 rows of the compiled data

```

5×12 DataFrame

Row	Date	HadCRUT_RawTemperature	HadCRUT_Temp	GISTEMP_RawTemperatu ...
-----	------	------------------------	--------------	--------------------------

	Date	Float64?	Float64?	Float64?	...
1	2025-11-01	1.0748	1.43471		1. ...
2	2025-12-01	0.924534	1.28444		1.
3	2026-01-01	0.997841	1.35775		1.
4	2026-02-01	1.04689	1.4068		1.
5	2026-03-01	missing	missing		1. ...

9 columns omitted

The compiled dataset is now saved here `Compiled_Global_Temperature_Data.csv`. The data contains the date, raw temperature anomalies for each dataset, and the temperature anomalies relative to the 1850-1900 baseline. It also includes the ONI anomalies.

10 Plot the data

Loading the compiled data and setting plot aesthetics.

```
# Load the compiled data and plot packages

using Plots
compiled_data = CSV.read(compiled_filename, DataFrame)

# Set plot aesthetics
theme(:ggplot2)
default(
    fontfamily = "Computer Modern",
    tickfontsize = 10,          legendfontsize = 10,
    titlefontsize = 12,
    xlabelfontsize = 10,
    ylabelfontsize = 10,
    titlefontfamily = "Computer Modern",
    legendfontfamily = "Computer Modern",
    tickfontfamily = "Computer Modern"
)

# Extract the dates for x-axis ticks
# This will be used for the x-axis ticks in the plot
xls = compiled_data.Date;
```

Plotting the data.

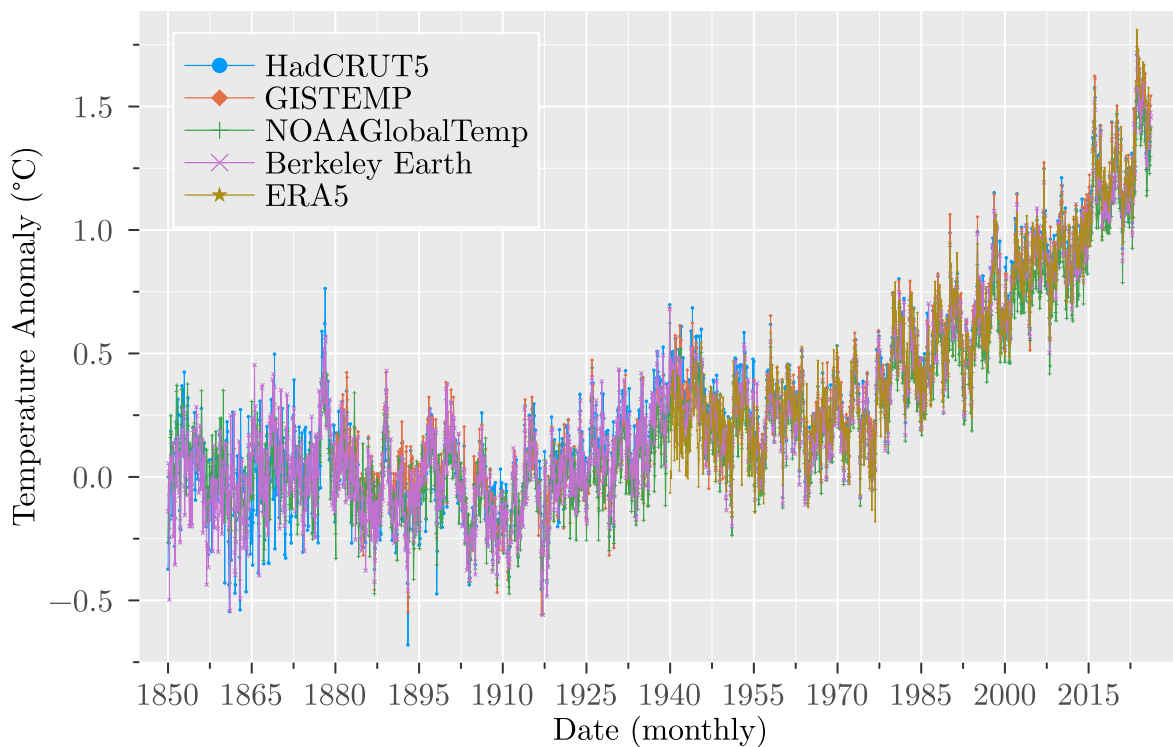
```
# Plot the data one dataset at a time, for clarity
p = plot(compiled_data.Date, compiled_data.HadCRUT_Temp, label="HadCRUT5", xlabel="Date
(monthly)", ylabel="Temperature Anomaly (°C)", title="Global Temperature Anomalies",
```

```

linewidth=0.5, markershape=:circle, markersize=1)
plot!(compiled_data.Date, compiled_data.GISTEMP_Temp, label="GISTEMP", xlabel="Date
(monthly)", ylabel="Temperature Anomaly (°C)", title="Global Temperature Anomalies" ,
linewidth=0.5, markershape=:diamond, markersize=1)
plot!(compiled_data.Date, compiled_data.NOAA_Temp, label="NOAAGlobalTemp",
xlabel="Date (monthly)", ylabel="Temperature Anomaly (°C)", title="Global Temperature
Anomalies" , linewidth=0.5, markershape=:+, markersize=1)
plot!(compiled_data.Date, compiled_data.Berkeley_Temp, label="Berkeley Earth",
xlabel="Date (monthly)", ylabel="Temperature Anomaly (°C)", title="Global Temperature
Anomalies", linewidth=0.5, markershape=:xcross, markersize=1)
plot!(compiled_data.Date, compiled_data.ERA5_Temp, label="ERA5", xlabel="Date
(monthly)", ylabel="Temperature Anomaly (°C)", title="Global Temperature Anomalies",
linewidth=0.5, markershape=:star, markersize=1)
plot!(legend=:topleft, xticks=(xls[1:180:end], Dates.format.(xls[1:180:end], "Y"))) #
Set x-axis ticks every 180 months
display(p)

```

Global Temperature Anomalies



The plot shows the global temperature anomalies for each dataset. The HadCRUT5 dataset is shown in blue, GISTEMP in orange, NOAAGlobalTemp in green, Berkeley Earth in purple, and ERA5 in brown. The x-axis represents the date (monthly), and the y-axis represents the temperature anomaly in degrees Celsius.


```
# Save the plot
```

```
savefig("data/Global_Temperature_Anomalies.pdf")  
savefig("data/Global_Temperature_Anomalies.png")
```

```
"/Users/eduardo/Library/CloudStorage/OneDrive-AalborgUniversitet/Research/CLIMATE/  
Global Temperature Anomalies/data/Global_Temperature_Anomalies.png"
```

The plot is saved as a PDF file as `Global_Temperature_Anomalies.pdf`.

10.1 The last 30 years

Zooming in on the last 30 years of data.

```
# Zoom in on the last 30 years of data
```

```
compiled_data_zoomed = compiled_data[compiled_data.Date .>= Date(1995, 1, 1), :]
```

```
p_zoomed = plot(compiled_data_zoomed.Date, compiled_data_zoomed.HadCRUT_Temp,  
label="HadCRUT5", xlabel="Date (monthly)", ylabel="Temperature Anomaly (°C)",  
title="Global Temperature Anomalies (Last 30 Years)", linewidth=0.5,  
markershape=:circle, markersize=1)
```

```
plot!(compiled_data_zoomed.Date, compiled_data_zoomed.GISTEMP_Temp, label="GISTEMP",  
xlabel="Date (monthly)", ylabel="Temperature Anomaly (°C)", title="Global Temperature  
Anomalies (Last 30 Years)", linewidth=0.5, markershape=:diamond, markersize=1)
```

```
plot!(compiled_data_zoomed.Date, compiled_data_zoomed.NOAA_Temp,  
label="NOAAGlobalTemp", xlabel="Date (monthly)", ylabel="Temperature Anomaly (°C)",  
title="Global Temperature Anomalies (Last 30 Years)", linewidth=0.5, markershape=:+,  
markersize=1)
```

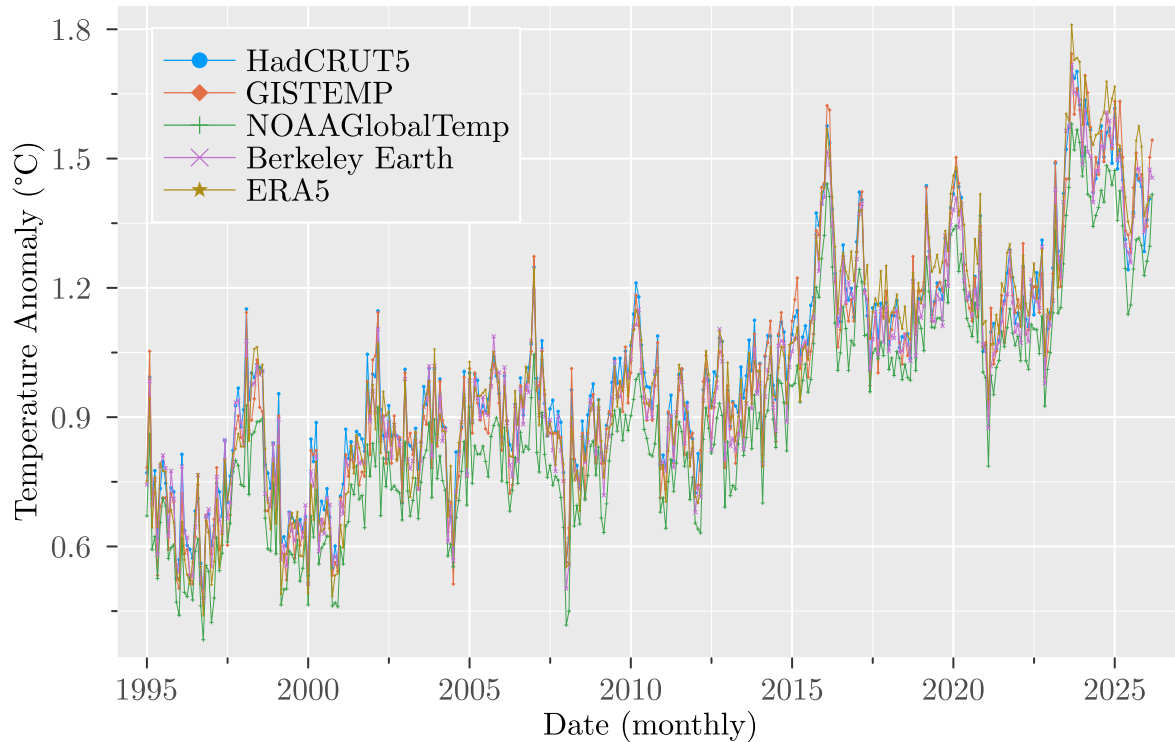
```
plot!(compiled_data_zoomed.Date, compiled_data_zoomed.Berkeley_Temp, label="Berkeley  
Earth", xlabel="Date (monthly)", ylabel="Temperature Anomaly (°C)", title="Global  
Temperature Anomalies (Last 30 Years)", linewidth=0.5, markershape=:xcross,  
markersize=1)
```

```
plot!(compiled_data_zoomed.Date, compiled_data_zoomed.ERA5_Temp, label="ERA5",  
xlabel="Date (monthly)", ylabel="Temperature Anomaly (°C)", title="Global Temperature  
Anomalies (Last 30 Years)", linewidth=0.5, markershape=:star, markersize=1)
```

```
plot!(legend=:topleft, xticks=(compiled_data_zoomed.Date[1:60:end], Dates.format.  
(compiled_data_zoomed.Date[1:60:end], "Y")))
```

```
display(p_zoomed)
```

Global Temperature Anomalies (Last 30 Years)



The zoomed-in plot shows the global temperature anomalies for each dataset over the last 30 years. The HadCRUT5 dataset is shown in blue, GISTEMP in orange, NOAAGlobalTemp in green, Berkeley Earth in purple, and ERA5 in brown. The x-axis represents the date (monthly), and the y-axis represents the temperature anomaly in degrees Celsius.

```
# Save the zoomed plot
```

```
savefig("data/Global_Temperature_Anomalies_Last_30_Years.pdf")  
savefig("data/Global_Temperature_Anomalies_Last_30_Years.png")
```

```
"/Users/eduardo/Library/CloudStorage/OneDrive-AalborgUniversitet/Research/CLIMATE/  
Global Temperature Anomalies/data/Global_Temperature_Anomalies_Last_30_Years.png"
```

The plot is saved as a PDF file as `Global_Temperature_Anomalies_Last_30_Years.pdf`.

10.2 Adding El Niño and La Niña events

To add the periods of El Niño and La Niña events to the plot, we will use the SST anomalies. An El Niño event is defined as a period when the SST anomaly is above $+0.5^{\circ}\text{C}$, while a La Niña event is defined as a period when the SST anomaly is below -0.5°C .

First, we need to classify the SST anomalies.

```

# Classify SST anomalies into El Niño, La Niña, and Neutral events

ONI_Anomaly = compiled_data_zoomed.ONI_Anomaly[.!
ismissing.(compiled_data_zoomed.ONI_Anomaly)]

T = length(ONI_Anomaly)
indicator = zeros{Int, T}

for i in 1:T
    if ONI_Anomaly[i] >= 0.5
        indicator[i] = 1
    elseif ONI_Anomaly[i] <= -0.5
        indicator[i] = -1
    else
        indicator[i] = 0
    end
end
end

```

Then we can add the El Niño and La Niña events to the plot. The shaded areas will indicate the periods of El Niño (red) and La Niña (blue) events based on the SST anomalies.

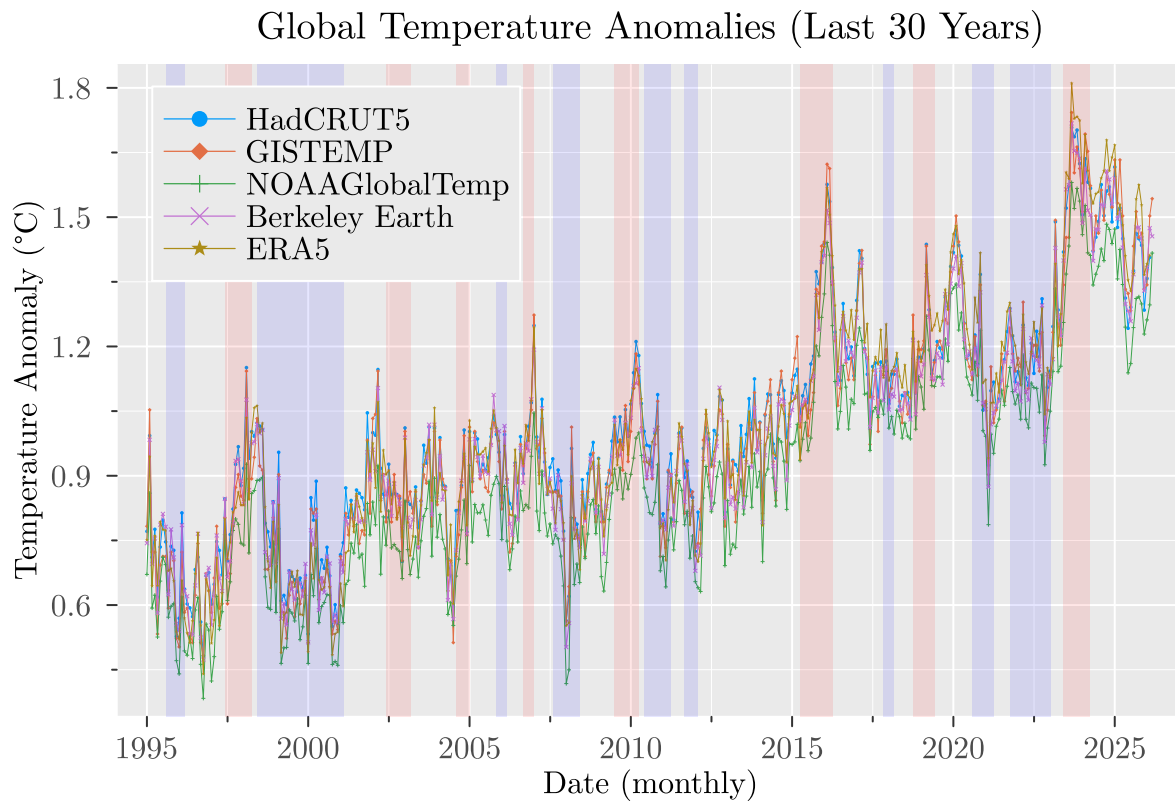
```

p_zoomed_oni = p_zoomed

i = 1
while i <= length(indicator)
    current_val = indicator[i]
    if current_val in (-1, 1)
        start_idx = i
        while i <= length(indicator) && indicator[i] == current_val
            i = i + 1
        end
        stop_idx = i - 1
        if (stop_idx <= start_idx) || (stop_idx - start_idx < 4)
            continue
        else
            vspan!(p_zoomed_oni, [compiled_data_zoomed.Date[start_idx],
compiled_data_zoomed.Date[stop_idx]], color=current_val == 1 ? :red : :blue, alpha=0.1,
label = "")
        end
    else
        i = i + 1
    end
end
end

```

```
display(p_zoomed_oni)
```



The plot now shows the global temperature anomalies for each dataset over the last 30 years, with shaded areas indicating El Niño (red) and La Niña (blue) events based on the ONI anomalies. The x-axis represents the date (monthly), and the y-axis represents the temperature anomaly in degrees Celsius.

```
# Save the plot with ONI events
```

```
savefig("data/Global_Temperature_Anomalies_Last_30_Years_ONI.pdf")
```

```
savefig("data/Global_Temperature_Anomalies_Last_30_Years_ONI.png")
```

```
"/Users/eduardo/Library/CloudStorage/OneDrive-AalborgUniversitet/Research/CLIMATE/  
Global Temperature Anomalies/data/Global_Temperature_Anomalies_Last_30_Years_ONI.png"
```

The plot is saved as a PDF file as `Global_Temperature_Anomalies_Last_30_Years_ONI.pdf`.

11 References

12 Citation

If you use any of the data or code in this notebook, please cite the original datasets and this notebook as follows:

```
@article{vera-valdes2026,  
  author = {Vera-Valdés, J. Eduardo},  
  title = {Data visualization and comparison of global temperature anomalies from  
multiple datasets},  
  journal = {Submitted},  
  year = {2026},  
}
```

Bibliography

- [1] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, “Julia: A fresh approach to numerical computing,” *SIAM review*, vol. 59, no. 1, pp. 65–98, 2017, [Online]. Available: <https://doi.org/10.1137/141000671>
- [2] C. P. Morice *et al.*, “An updated assessment of near-surface temperature change from 1850: The HadCRUT5 data set,” *Journal of Geophysical Research: Atmospheres*, vol. 126, no. 3, p. e2019JD032361, 2021.
- [3] GISTEMP, “GISS Surface Temperature Analysis (GISTEMP), version 4.” [Online]. Available: <https://data.giss.nasa.gov/gistemp/>
- [4] B. Huang, X. Yin, M. J. Menne, R. S. Vose, and H.-M. Zhang, “NOAA Global Surface Temperature Dataset (NOAAGlobalTemp).” [Online]. Available: <https://doi.org/10.25921/rzxg-p717>
- [5] R. A. Rohde and Z. Hausfather, “The Berkeley Earth land/ocean temperature record,” *Earth System Science Data*, vol. 12, no. 4, pp. 3469–3479, 2020, doi: 10.5194/essd-12-3469-2020.
- [6] H. Hersbach *et al.*, “ERA5 monthly averaged data on single levels from 1940 to present.” [Online]. Available: <https://doi.org/10.24381/cds.f17050d7>
- [7] B. Huang *et al.*, “Extended Reconstructed Sea Surface Temperature, Version 6 (ERSSTv6). Part I: An Artificial Neural Network Approach,” *Journal of Climate*, vol. 38, no. 4, pp. 1105–1121, 2025, doi: 10.1175/JCLI-D-23-0707.1.
- [8] B. Huang *et al.*, “Extended Reconstructed Sea Surface Temperature, Version 6 (ERSSTv6). Part II: Upgrades on Quality Control and Large-Scale Filter,” *Journal of Climate*, vol. 38, no. 4, pp. 1123–1136, 2025, doi: 10.1175/JCLI-D-24-0185.1.